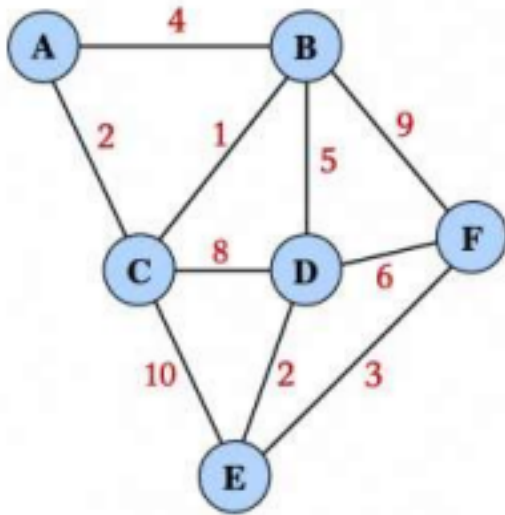


แบบฝึกหัด Dijkstra's Algorithm, Minimum Spanning Tree, Kruskal, Prim และ Borůvka

ข้อ 1: Dijkstra's Algorithm — Shortest Path

กำหนด Weighted Undirected Graph ดังนี้



กำหนด Edge ดังนี้

Edge	Weight
A-B	4
A-C	2
B-C	1
B-D	5
C-D	8
C-E	10
D-E	2
D-F	6
E-F	3
B-F	9

ให้ใช้ Dijkstra's Algorithm โดยเริ่มต้นที่ Vertex A

1.1 จงแสดงขั้นตอนของ Dijkstra ที่ละรอบ โดยระบุ

- Vertex ที่ถูกเลือก
- ค่า Distance ของ A, B, C, D, E และ F
- การ Relaxation ที่เกิดขึ้น

$$A = 0$$

$$B = \infty$$

$$C = \infty$$

$$D = \infty$$

$$E = \infty$$

$$F = \infty$$

**รอบที่ 1 → เลือก A
จาก A**

$$A \rightarrow B = 0 + 4 = 4$$

$$A \rightarrow C = 0 + 2 = 2$$

ดังนั้น

$$B = 4$$

$$C = 2$$

Relaxation: $A \rightarrow B$, $A \rightarrow C$

**รอบที่ 2 → เลือก C
เพราะ $C = 2$ น้อยที่สุด
ตรวจ**

$$C \rightarrow B = 2 + 1 = 3 < 4$$

จึงเปลี่ยน $B = 3$

$$C \rightarrow D = 2 + 8 = 10$$

$$C \rightarrow E = 2 + 10 = 12$$

จึงได้ $D = 10$ และ $E = 12$

Relaxation: $C \rightarrow B$, $C \rightarrow D$, $C \rightarrow E$

รอบที่ 3 → เลือก B

เพราะ $B = 3$

$$B \rightarrow D = 3 + 5 = 8 < 10$$

จึง $D = 8$

$$B \rightarrow F = 3 + 9 = 12$$

จึง $F = 12$

Relaxation: $B \rightarrow D$, $B \rightarrow F$

รอบที่ 4 → เลือก D

$D = 8$

$D \rightarrow E = 8 + 2 = 10 < 12$

ดังนั้น $E = 10$

$D \rightarrow F = 8 + 6 = 14 > 12$

ไม่เปลี่ยน F

Relaxation: $D \rightarrow E$

รอบที่ 5 → เลือก E

$E \rightarrow F = 10 + 3 = 13$

แต่ F เดิม = 12

$13 > 12$

จึงไม่เปลี่ยน

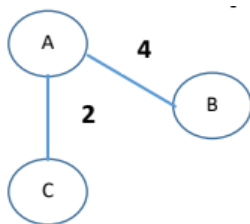
ไม่มี Relaxation

1

(Relaxation ใน Dijkstra's Algorithm คือกระบวนการ ตรวจสอบว่าเราพบเส้นทางที่สั้นกว่าเดิมไปยัง โหนดหนึ่งหรือไม่ ถ้าพบ ก็ปรับค่า Distance ของโหนดนั้นให้เล็กลง)

Relaxation = “ลองเส้นทางใหม่แล้วถ้าสั้นกว่าเดิมก็Update”

ตัวอย่าง สมมติว่ากำลังอยู่ที่โหนด A



ตอนเริ่มต้นกำหนด

$\text{Distance}(A) = 0$

$\text{Distance}(B) = \infty$

$\text{Distance}(C) = \infty$

เมื่อพิจารณาเส้นทาง $A \rightarrow B$

$\text{newDistance} = \text{Distance}(A) + \text{Weight}(A,B)$

$= 0 + 4$

$= 4$

เมื่อเปรียบเทียบกับค่าเดิมของ B เดิม $\text{Distance}(B) = \infty$

ใหม่ $= 4$ เพราะ $4 < \infty$ จึง Relax ค่า B เป็น

$\text{Distance}(B) = 4$

สูตรหลักของ Relaxation คือ

$\text{newDistance} = \text{dist}[u] + \text{weight}(u,v)$ จากนั้น

ตรวจว่า

$$\text{dist}[u] + \text{weight}(u,v) < \text{dist}[v]$$

ถ้าเป็นจริง ให้เปลี่ยน

$\text{dist}[v] = \text{dist}[u] + \text{weight}(u,v)$ ใน Java จะ

เขียนแนวคิดประมาณนี้:

```
int newDistance = distance[u] + weight;
```

```
if (newDistance < distance[v]) {  
    distance[v] = newDistance;  
}
```

2

จากข้อ 1 ให้ นักศึกษาเติมตาราง

รอบ	Vertex ที่เลือก	A	B	C	D	E	F
Initial	-	0	∞	∞	∞	∞	∞
1	A	0	4	2	∞	∞	∞
2	C	0	3	2	10	12	∞

3	B	0	3	2	8	12	12
4	D	0	3	2	8	10	12
5	E	0	3	2	8	10	12
6	F	0	3	2	8	10	12

1.2 จงหา Shortest Path จาก A ไปยัง

- D
- E
- F

พร้อมระบุ Total Weight

1) จาก A ไป D

$A \rightarrow C \rightarrow B \rightarrow D$

น้ำหนัก

$$2 + 1 + 5 = 8$$

$A \rightarrow C \rightarrow B \rightarrow D$, Total Weight = 8

2) จาก A ไป E

$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E$

น้ำหนัก

$$2 + 1 + 5 + 2 = 10$$

3) จาก A ไป F

$A \rightarrow C \rightarrow B \rightarrow F$

น้ำหนัก

$$2 + 1 + 9 = 12$$

$A \rightarrow C \rightarrow B \rightarrow F$, Total Weight = 12

1.3 สมมติว่า Edge E-F ถูกเปลี่ยนจากน้ำหนัก 3 เป็น -5 อธิบายว่าเหตุใดจึงไม่ควรใช้ Dijkstra's Algorithm กับกราฟดังกล่าว

ไม่ควรใช้ Dijkstra เพราะ Dijkstra ไม่รองรับ Edge ที่มีน้ำหนัก

ติดลบ เนื่องจาก Distance ที่เคยเลือกไปแล้วอาจถูกทำให้มีค่าน้อยลงภายหลัง ส่งผลให้คำตอบผิดได้

Dijkstra $\rightarrow$ Weight ต้องไม่ติดลบ

1.4 วิเคราะห์ Java Code

```
for (Edge edge : graph[u]) {  
    int v = edge.to;  
    int weight = edge.weight;  
  
    if (!visited[v] &&  
        dist[u] + weight < dist[v]) {  
  
        dist[v] = dist[u] + weight;  
        parent[v] = u;  
    }  
}
```

ตอบคำถามต่อไปนี้

ก. เงื่อนไข **$\text{dist}[u] + \text{weight} < \text{dist}[v]$** มีความหมายอย่างไรใน Dijkstra?
ตรวจสอบว่าเส้นทางใหม่จาก u ไป v มีระยะทางสั้นกว่าค่าเดิมหรือไม่ ถ้าสั้นกว่าให้ Update ค่าใหม่

$\text{dist}[v] = 8$

ข. **$\text{parent}[v] = u$** มีไว้เพื่ออะไร?

ใช้เก็บว่า

Vertex v เดินทางมาจาก Vertex u

เพื่อใช้ย้อนกลับหา **Shortest Path**

ค. หากลบเงื่อนไข **!visited[v]** ออก ผลกระทบที่อาจเกิดขึ้นคืออะไร?
อาจทำให้ Vertex ที่ถูกเลือกและประมวลผลไปแล้วถูก Update อีกครั้ง
ทำให้ Algorithm ทำงานผิดพลาดของ Dijkstra ได้

3

ข้อ 2: Kruskal's Algorithm

ใช้ Graph เดียวกับข้อ 1 จงหา Minimum Spanning Tree (MST) ด้วย Kruskal's Algorithm 2.1 เรียง Edge ทั้งหมดจาก Weight น้อยไปมาก

ลำดับ Weight	Edge	
1	B-C	1
2	A-C	2
3	D-E	2
4	E-F	3
5	A-B	4
6	B-D	5
7	D-F	6
8	C-D	8
9	B-F	9
10	C-E	10

2.2 พิจารณา Edge ตามลำดับ และระบุว่า

- Accept — เลือกเข้า MST
- Reject — ไม่เลือก เพราะทำให้เกิด Cycle

สร้างตาราง

ลำดับ	Edge	Weight	Accept/Reject	เหตุผล
-------	------	--------	---------------	--------

1	B-C	1	Accept	ไม่เกิด Cycle
2	A-C	2	Accept	ไม่เกิด Cycle
3	D-E	2	Accept	ไม่เกิด Cycle
4	E-F	3	Accept	ไม่เกิด Cycle
5	A-B	4	Reject	เกิด Cycle
6	B-D	5	Accept	ไม่เกิด Cycle

หยุดเมื่อ MST มีจำนวน Edge ครบตามที่ต้องการ

2.3 ว่าด MST ที่ได้ และหำนำหนักรวม

MST คือ

B-C = 1

A-C = 2

D-E = 2

E-F = 3

B-D = 5

วาดได้ว่า

A —2— C —1— B —5— D —2— E —3— F

รวม

$1 + 2 + 2 + 3 + 5$

= 13

MST Total Weight = 13

2.4 ถ้ากราฟมี V Vertices จำนวน Edge ใน MST ต้องเป็นเท่าใด และเพราะเหตุใด?

V - 1 Edge

เพราะ Tree ที่มี V Vertex จะต้องใช้ V-1 Edge เพื่อเชื่อมทุก Vertex โดยไม่เกิด Cycle

ตัวอย่างนี้

$$V = 6$$

$$6 - 1 = 5$$

ดังนั้นมี **5 Edge**

2.5 วิเคราะห์ Java Code

```
for (Edge edge : edges) {  
  
    int rootU = find(edge.u);  
    int rootV = find(edge.v);  
  
    if (rootU != rootV) {  
        mst.add(edge);  
        union(rootU, rootV);  
    }  
}
```

ก. อธิบายว่า **rootU != rootV** ช่วยป้องกันปัญหาอะไร?

ตรวจสอบว่า U และ V อยู่คนละ Component กันหรือไม่ ถ้าอยู่คนละ Component สามารถเพิ่ม Edge ได้โดยไม่เกิด Cycle

ข. และถ้า **rootU == rootV** แล้วเรายังเพิ่ม Edge เข้า MST จะเกิดอะไรขึ้น?

U และ V อยู่ Component เดียวกันแล้ว ถ้าเพิ่ม Edge จะทำให้เกิด Cycle

ดังนั้น Kruskal จึงไม่เลือก Edge นี้

4

ข้อ 3: Prim's Algorithm และการเปรียบเทียบกับ Kruskal
ใช้ Graph เดียวกับข้อ 1 ให้หา MST ด้วย Prim's Algorithm โดยเริ่มจาก Vertex A 3.1 แสดง Edge ที่ Prim เลือกในแต่ละรอบ

รอบ	Vertices ใน MST	Candidate Edges ที่สำคัญ	Edge ที่เลือก	Weight
1	(A)	A-C(2), A-B(4)	A-C	2
2	A,C	C-B(1), A-B(4), C-D(8), C-E(10)	C-B	1
3	A,B,C	B-D(5), B-F(9), C-D(8), C-E(10)	B-D	5
4	A,B,C,D	D-E(2), D-F(6), B-F(9), C-E(10)	D-E	2
5	A,B,C,D,E	E-F(3), D-F(6), B-F(9)	E-F	3

3.2 ว่าด MST ที่ได้และหาค่าหน้าหกรวม

A —2— C —1— B —5— D —2— E —3— F

รวม

$$2 + 1 + 5 + 2 + 3 = 13$$

Total Weight = 13

3.3 เปรียบเทียบแนวคิดของ Prim กับ Kruskal โดยอธิบายว่า

- Algorithm ใดเลือก Edge ที่เล็กที่สุดของทั้ง Graph
- Algorithm ใดเลือก Edge ที่เล็กที่สุดที่เชื่อม Tree

ปัจจุบันกับ Vertex ภายนอก

ข้อ 1 → Kruskal

เพราะ Kruskal เรียง Edge ทั้ง Graph จากนั้นน้อยไปมาก แล้วเลือกทีละเส้น

ข้อ 2 → Prim

เพราะ Prim เริ่มจาก Vertex แล้วขยาย Tree ออกไป โดยเลือก Edge ที่เล็กที่สุดที่เชื่อมกับ Vertex ภายนอก

3.4 นักศึกษาคนหนึ่งกล่าวว่า

“Prim และ Dijkstra เหมือนกัน เพราะทั้งคู่เลือกค่าน้อยที่สุดในแต่ละรอบ ดังนั้นผลลัพธ์ต้องเป็น Tree เดียวกัน”

จงวิเคราะห์ว่าข้อความนี้ถูกหรือผิด พร้อมอธิบายเหตุผล

ผิด

เพราะทั้งสอง Algorithm มีเป้าหมายต่างกัน

Dijkstra

หา

Shortest Path จาก Source ไปยัง Vertex ต่าง ๆ

Prim

หา

Minimum Spanning Tree ที่มีน้ำหนักรวมต่ำที่สุด

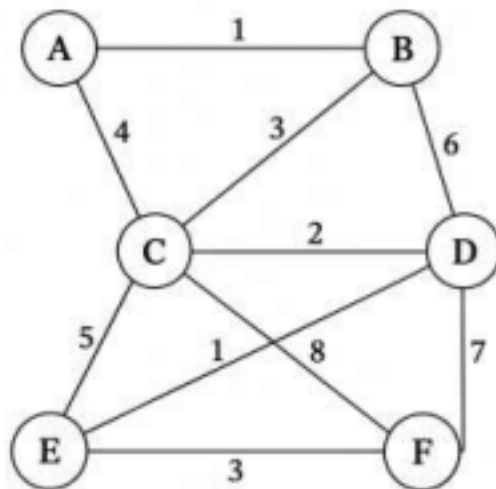
ดังนั้นแม้ทั้งคู่เลือกค่าน้อยที่สุด แต่ เกณฑ์ที่ใช้เลือกไม่เหมือนกัน

Dijkstra = หาเส้นทางสั้นที่สุด

Prim = สร้าง MST

ข้อ 4: Borůvka's Algorithm

Question 4: Boruvka's Algorithm



เริ่มต้นให้แต่ละ Vertex เป็น Component ของตนเอง {A} {B} {C} {D} {E} {F}

4.1 ใน Boruvka รอบที่ 1 ให้หา Cheapest Outgoing Edge ของแต่ละ Component

Component	Cheapest Edge	Weight
{A}	A-C	2
{B}	B-C	1
{C}	C-B	1
{D}	D-E	2
{E}	E-D	2
{F}	F-E	3

จากนั้นระบุ Components ใหม่หลังจาก Merge

เลือก Edge ที่ไม่ซ้ำกัน

$$A-C = 2$$

$$B-C = 1$$

$$D-E = 2$$

$$E-F = 3$$

Merge แล้วได้

{A,B,C}

{D,E,F}

4.2 ดำเนินการ Borůvka รอบที่ 2 จนได้ MST
ให้ระบุ

- Components ก่อนเริ่มรอบ
- Cheapest outgoing edge ของแต่ละ Component
- Edge ที่ถูกเพิ่มเข้า MST

ก่อนเริ่ม

{A,B,C}

{D,E,F}

Edge ที่เชื่อมสอง Component ได้แก่

$$B-D = 5$$

$$C-D = 8$$

$$B-F = 9$$

$$C-E = 10$$

Edge ที่ถูกที่สุดคือ

$$B-D = 5$$

ดังนั้นเพิ่ม

$$\mathbf{B-D = 5}$$

Components รวมเป็น

{A,B,C,D,E,F}

ครบทุก Vertex

MST ของ Borůvka

$$A-C = 2$$

$$B-C = 1$$

$$D-E = 2$$

$$E-F = 3$$

$$B-D = 5$$

รวม

$$2 + 1 + 2 + 3 + 5$$

$$= 13$$

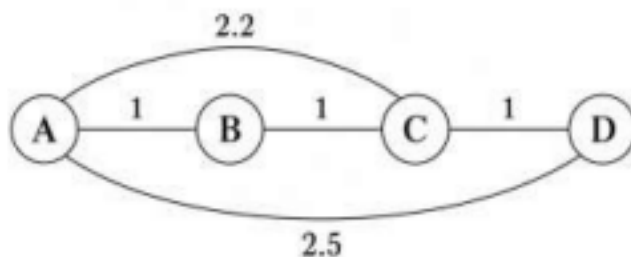
ตอบ

MST Total Weight = 13

ข้อ 5: วิเคราะห์ Shortest Path vs Minimum Spanning Tree

มหาวิทยาลัยต้องการเชื่อมต่ออาคาร 4 อาคารด้วย Network Cable กำหนด Graph

Question 5: Shortest Path vs MST



สถานการณ์ A

มหาวิทยาลัยต้องการเชื่อมทุกอาคารเข้าด้วยกันโดยใช้สาย Network รวมให้น้อยที่สุด

5.1 ควรใช้ Algorithm ประเภทใดระหว่าง Dijkstra หรือ MST Algorithm พร้อมอธิบายเหตุผล

ใช้ Minimum Spanning Tree (MST)

โจทย์ต้องการเชื่อมอาคารทั้ง 4 อาคารเข้าด้วยกัน โดยใช้สาย Network รวมให้สั้นที่สุด จึงใช้ MST เพราะ MST จะเลือกเส้นทางที่ทำให้ อาคารทุกอาคารเชื่อมถึงกัน และมี Total Weight น้อยที่สุด

5.2 หา MST และคำนวณ Total Weight
เรียง Edge จากค่าน้อยไปมาก

1. A-B = 1 

2. B-C = 1 

3. C-D = 1 

4. A-C = 2.2  ไม่เลือก เพราะจะเกิด Cycle

5. A-D = 2.5  ไม่เลือก เพราะจะเกิด Cycle

ดังนั้น MST คือ

A-B, B-C, C-D

คำนวณ Total Weight: $1+1+1=3$

MST = {A-B, B-C, C-D}

Total Weight = 3

ภาพที่ควรนึก:

A —1— B —1— C —1— D

สถานการณ์ B

กำหนดให้ A เป็น Server หลัก ต้องการหา เส้นทางจาก A ไป D ที่มี Cost ต่ำที่สุด

5.3 ใช้ Dijkstra หรือการวิเคราะห์ Shortest Path หาเส้นทางที่สั้นที่สุดจาก A ไป D

ใช้ Dijkstra Algorithm

เพราะโจทย์นี้ต้องการหา Shortest Path จากจุดเริ่มต้น A ไปยัง D

ลองดูเส้นทางที่เป็นไปได้:

ทางที่ 1

$A \rightarrow B \rightarrow C \rightarrow D$

$$1+1+1=3$$

ทางที่ 2

$A \rightarrow C \rightarrow D$

$$2.2+1=3.2$$

ทางที่ 3

$A \rightarrow D$

$$2.5$$

ดังนั้นเส้นทางที่สั้นที่สุดคือ

$A \rightarrow D$

$$\text{Cost} = 2.5$$

ใช้ Dijkstra Algorithm

Shortest Path = $A \rightarrow D$

$$\text{Cost} = 2.5$$

5.4 ถ้าใช้เฉพาะ Edge ใน MST จากข้อ 5.2 เส้นทางจาก A ไป D มี Cost เท่าใด? เปรียบเทียบกับ Shortest Path จากข้อ 5.3

MST ที่เราได้คือ

$A \text{ --- } 1 \text{ --- } B \text{ --- } 1 \text{ --- } C \text{ --- } 1 \text{ --- } D$

ถ้าต้องเดินจาก A ไป D โดยใช้เฉพาะ Edge ที่อยู่ใน MST

จะได้:

$A \rightarrow B \rightarrow C \rightarrow D$

คำนวณ Cost:

$$1+1+1=3$$

ดังนั้น

Cost จาก MST = 3

วิธี	เส้นทาง A → D	Co st
MST	$A \rightarrow B \rightarrow C$ → D	3
Shortest Path	$A \rightarrow D$	2.5

ดังนั้น

MST Path มี Cost = 3

Shortest Path มี Cost = 2.5

ต่างกัน:

$$3-2.5=0.5$$

5.5 จากผลที่ได้ จงวิเคราะห์ข้อความต่อไปนี้“Minimum Spanning Tree ทำให้เส้นทางจาก Source ไป ทุก Vertex สั้นที่สุดเสมอ” ถูกหรือผิด เพราะเหตุใด?

เพราะ MST มีเป้าหมายเพื่อลดน้ำหนักรวมของ Edge ทั้งหมดในต้นไม้ ไม่ได้มีเป้าหมายเพื่อทำให้ระยะทางจาก Source ไปแต่ละ Vertex สั้นที่สุด

ส่วน Dijkstra มีเป้าหมายเพื่อหา Shortest Path จาก Source ไปยัง Vertex อื่น ๆ

ตัวอย่างจากข้อนี้:

MST ได้

A —1— B —1— C —1— D

ทำให้ $A \rightarrow D$ มี Cost

$$1+1+1=3$$

แต่ใน Graph จริงมี Edge

A —2.5— D

ดังนั้น Shortest Path คือ

$$A \rightarrow D = 2.5$$

ซึ่งสั้นกว่า 3

